CONSTITUTIONAL_COMPLIANCE_REPOR

Helix Cluster OS — Constitutional Compliance Report

Field	Value
Document ID	CCR-001
Revision	1.0
Date	2026-03-04
Classification	INVESTIGATION — INTERNAL
Authors	Autonomous Compliance Agent
Status	FINAL

Field	Value
Scope	All constitutional rules from Constitution.md, CLAUDE.md, AGENTS.md, and Challenges CONSTITUTION.md

---

Table of Contents

Chapter 1: Constitution Rules Extraction

1.1 Constitution.md Rules (PCS-1 through PCS-6.6)

PCS-1: Cross-Platform Parity

Rule: Every feature must have implementations for Linux, macOS, and Windows/WSL2. No OS may run on a mock/stub for a feature that claims real operation.

PCS-2: GPU Backend Coverage

Rule: All 4 vendor backends (NVIDIA, AMD, Apple, Intel) must have real-hardware testing.

PCS-3: No Hardcoded Secrets

Rule: Credentials must never be tracked in source control. Pre-store leak audit per §11.4.10.A.

PCS-4: Container Mandate

Rule: All services must be containerizable. No sudo/interactive/root-prompting processes.

PCS-5: Subagent-Driven Development

Rule: Development is subagent-driven by default per §11.4.20.

PCS-6: Fetch-Before-Edit

Rule: Always fetch latest state before editing per §11.4.37.

PCS-6.1: No Orphaned Code

Rule: All code must be reachable from at least one binary. Orphaned code must be triaged.

PCS-6.2: Coverage Gate

Rule: 80% line coverage must be enforced via `pkg/covgate`.

PCS-6.3: Quality Gate

Rule: Quality metrics must meet baseline thresholds per `test/qualitygate/baseline_metrics.json`.

PCS-6.4: Phase Gate

Rule: Each phase must pass all gate criteria before proceeding.

PCS-6.5: Architecture Lint

Rule: Layer violations must be detected and prevented by `pkg/archlint`.

PCS-6.6: etcd Lint

Rule: etcd key patterns must be validated by `pkg/etcdlint`.

1.2 CLAUDE.md Rules

CLAUDE-1: End-User Usability Guarantee

Rule Text (verbatim): > “We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!”

Operative Rules: 1. Tests are necessary but NOT sufficient — must verify unit + integration + E2E + Challenge 2. A test that passes on a non-functional feature is a PASS-bluff (§7.1 violation) 3. Every test MUST prove the feature works for end users 4. Challenges are bound equally — Challenge PASS on broken feature = defect 5. No mock-only validation for

real-world features 6. Sink-side evidence required before declaring feature complete

CLAUDE-2: Cross-Platform Parity Guarantee

Rule Text (verbatim): > “For any Linux specific technology we MUST implement for other platforms (macOS) proper equivalents so on every OS we use proper system equivalents!”

Operative Rules: 1. No OS may run on a mock/stub for a feature that claims real operation 2. Platform code split by build tags behind ONE shared interface 3. Tests prove the feature on the HOST OS — never blanket-skip non-Linux 4. Known Linux-specific hotspots requiring macOS equivalents documented 5. Applies retroactively and going forward

CLAUDE-3: Documentation & Materials Continuous-Sync Guarantee

Rule Text (verbatim): > “while we continuously change the project, by extending it, implementing new features, applying fixes and changes, whatever we do on project or any of its services, components, architecture or anything else MUST trigger proper updateing of main README document, all project documentation, user guides, manuals, webiste(s) (if any), diagrams, graphs, schemes, SQL defintions and all other related materials!”

Operative Rules: 1. Every change triggers a documentation pass — no exceptions 2. docs_chain is the mechanical enforcer — out of the box, never skipped 3. No escape hatch — no `--skip-docs-chain` 4. Coverage is complete and current 5. Runs regularly, not just at release 6. Standing parallel work stream

1.3 AGENTS.md Rules

AGENT-1: End-User Usability Guarantee

Rule: Same as CLAUDE-1 but binding on coding agents specifically.

AGENT-2: Documentation & Materials Continuous-Sync Guarantee

Rule: Same as CLAUDE-3 but binding on coding agents specifically.

1.4 QWEN.md Rules

QWEN-1: End-User Usability Guarantee

Rule: Same as CLAUDE-1 but binding on Qwen-specific agents.

QWEN-2: Documentation & Materials Continuous-Sync Guarantee

Rule: Same as CLAUDE-3 but binding on Qwen-specific agents.

1.5 Challenges CONSTITUTION.md Rules

CONST-033: Host Power Management Forbidden

Rule: No Challenge, test, or implementation may use host power management as a test mechanism.

CONST-035: Anti-Bluff Covenant

Rule: No test or Challenge may be designed to pass on non-functional code.

CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage

Rule: No fake or mock implementations beyond unit tests. 100% test type coverage required.

CONST-051: Evidence Taxonomy

Rule: All test and challenge evidence must follow a defined taxonomy.

Chapter 2: Compliance Assessment

2.1 PCS-1: Cross-Platform Parity

Status: ⚠️ PARTIAL

Evidence of compliance: - `pkg/resources/proc_linux.go` + `pkg/resources/proc_darwin.go` — Real platform-specific readers - `pkg/resources/cgroup_v2.go` — Linux cgroup support - `pkg/wireguard/netstack_darwin.go` — macOS WireGuard userspace - `internal/gpu/detect_linux.go` + `internal/gpu/detect_darwin.go` — GPU detection - `internal/health/syscall_unix.go` + `internal/health/syscall_windows.go` — Health syscalls - `pkg/powergater/reader_linux.go` + `pkg/powergater/reader_darwin.go` — Power management - `pkg/edgeheartbeat/reader_linux.go` + `pkg/edgeheartbeat/reader_darwin.go` — Edge heartbeat

Evidence of violation: - `pkg/resources/drm_other.go` — Stub for non-Linux DRM/GPU - `pkg/resources/accel_other.go` — Stub for non-Linux accelerators - `pkg/resources/proc_mock.go` — Mock for non-Linux /proc - No Windows/WSL2 implementations for any package - `internal/console/linux_boot.go` — Linux-specific boot, no macOS equivalent - `pkg/edgeheartbeat/e2e_test.go` — E2E test only on Linux

Specific files:

File	Issue	Severity
`pkg/resources/drm_other.go`	Returns empty GPU info on non-Linux	HIGH
`pkg/resources/accel_other.go`	No accelerator monitoring on non-Linux	MEDIUM
`pkg/resources/proc_mock.go`	Mock implementation for /proc	HIGH
`pkg/gpu/detect_other.go`	No GPU detection on non-Linux/macOS	HIGH
`cmd/helix-setup/mem_other.go`	Generic memory detection placeholder	LOW

Remediation plan: 1. Replace `drm_other.go` with real macOS GPU detection via IOKit 2. Replace `accel_other.go` with real macOS accelerator detection 3. Remove `proc_mock.go` — use real `proc_darwin.go` (already exists) 4. Add Windows/WSL2 implementations for key packages 5. Document Linux-only features with CLAUDE-2 justification

2.2 PCS-2: GPU Backend Coverage

Status: ❌ NON-COMPLIANT

Evidence of compliance: - `internal/gpu/backend.go` — Backend interface with 4 vendors defined - `internal/gpu/backend_darwin_apple.go` — Apple Metal backend - `internal/gpu/nvidia_parser.go` — NVIDIA SMI parser - `internal/gpu/nvidia_reader_linux.go` — NVIDIA reader

Evidence of violation: - No AMD ROCm backend implementation - No Intel oneAPI/SYCL backend implementation - Apple backend only tested on macOS - No real hardware testing for any backend except NVIDIA (partial)

Remediation plan: 1. Implement AMD ROCm backend (requires AMD GPU hardware) 2. Implement Intel oneAPI backend (requires Intel GPU

hardware) 3. Add integration tests for all 4 backends 4. Add GPU backend to CI pipeline (requires GPU runners)

2.3 PCS-3: No Hardcoded Secrets

Status: ✅ COMPLIANT

Evidence of compliance: - `DATABASE_URL` in Makefile uses variable with default (not a secret — dev DSN) - `SONAR_TOKEN` and `SNYK_TOKEN` referenced by name only, never stored - `.gitignore` includes common secret patterns - `scripts/run-migrations.sh` reads from environment, not hardcoded

Verification method:

```
# Scan for potential secrets
rg -i "password|secret|api_key|token.*=" --type go | grep
        -v "_test.go" | grep -v "vendor/"
# Result: No hardcoded secrets found in source
```

2.4 PCS-4: Container Mandate

Status: ⚠️ PARTIAL

Evidence of compliance: - `deploy/docker/helix_gateway.dockerfile` — Gateway container - `deploy/docker/helixd.dockerfile` — Main daemon container - `deploy/docker/helix_agent.dockerfile` — Agent container - `deploy/compose/helix_core.yml` — Core services compose - `deploy/compose/helix_infra.yml` — Infrastructure compose - `deploy/k8s/` — Kubernetes deployment manifests

Evidence of violation: - No Dockerfile for security, scheduler, session, build, health, policy, llm, advisory services - `docker_compose.yml` at root doesn't include all services - No container scanning (trivy) in build pipeline

Remediation plan: 1. Create Dockerfiles for all 14 services 2. Update docker_compose.yml with all services 3. Add container scanning to build pipeline 4. Ensure no sudo/interactive processes in containers

2.5 CLAUDE-1: End-User Usability Guarantee

Status: ❌ NON-COMPLIANT

Evidence of violation: - 178/212 pkg/ packages are orphaned — features exist but can't be used by end users - CRIT-1: JWT tokens not verified — authentication doesn't work - CRIT-2: Health service HTTP-only — gRPC health checking doesn't work - CRIT-3: Build service uses wrong imports — build orchestration bypassed - `cmd/helixd` is a hollow shell — core

daemon does nothing - Gateway routing incomplete — not all services accessible - 8 stub bluff packages — code claims to implement features but doesn't

Specific violations:

Violation	Feature	Impact	Severity
178 orphaned packages	Multiple subsystems	Features can't be used	CRITICAL
CRIT-1: JWT stubs	Authentication	Security bypass	CRITICAL
CRIT-2: Health HTTP-only	Health monitoring	Gateway can't use gRPC health	HIGH
CRIT-3: Build wrong imports	Build service	Orchestration bypassed	HIGH
Hollow helixd	Core daemon	No service orchestration	CRITICAL
Gateway incomplete	API access	Services unreachable	HIGH
8 stub bluffs	Multiple	Features non-functional	HIGH

Remediation plan: 1. Wire orphaned packages into binaries (Phase 1 of UNFINISHED_WORK_REPORT) 2. Fix JWT token verification (CRIT-1) 3. Add gRPC health service (CRIT-2) 4. Fix build imports (CRIT-3) 5. Implement helixd service orchestration 6. Complete gateway routing for all services 7. Replace stub bluff packages with real implementations

2.6 CLAUDE-2: Cross-Platform Parity Guarantee

Status: ⚠️ PARTIAL

Evidence of compliance: - Platform-specific files use build tags correctly (`_linux.go` , `_darwin.go` , `_other.go`) - `pkg/resources` has real implementations for both Linux and macOS - `pkg/wireguard` has `netstack darwin.go` for macOS userspace WireGuard - `internal/gpu` has `detect_linux.go` and `detect_darwin.go` - No blanket non-Linux skips found

Evidence of violation: - `pkg/resources/proc mock.go` exists — mock for non-Linux (should be removed) - `pkg/resources/drm_other.go` —

stub for non-Linux DRM - No Windows/WSL2 implementations - 7/13 WireGuard tests skipped on macOS/non-root

Remediation plan: 1. Remove `proc_mock.go` (real `proc_darwin.go` exists) 2. Implement real DRM/GPU detection for macOS 3. Add Windows/WSL2 implementations for key packages 4. Fix WireGuard test skips with mock wgctrl client

2.7 CLAUDE-3: Documentation & Materials Continuous-Sync Guarantee

Status: ⚠️ PARTIAL

Evidence of compliance: - `.docs_chain/contexts/tracked_docs.yaml` — Registered doc contexts - `.docs_chain/contexts/gap_audits.yaml` — Gap audit context - `scripts/docs/generate.sh` — Documentation generation script - `scripts/docs/verify.sh` — Documentation verification script - `scripts/docs/run_docs_chain.sh` — docs_chain execution - `scripts/docs/update_continuation.sh` — Continuation update - `scripts/docs/db_to_md.py` — Database to Markdown conversion

Evidence of violation: - Some documentation may be stale (not verified every wave) - Not all materials registered in `.docs_chain/contexts/` - Export formats (md → html/pdf/docx) not verified - `docs_chain verify` not enforced as pre-build gate

Remediation plan: 1. Register all materials in `.docs_chain/contexts/` 2. Add `docs_chain verify` to pre-build gate 3. Ensure all exports are generated and verified 4. Run docs_chain sync every wave

2.8 §1.1: Mutation-Paired Tests

Status: ❌ NON-COMPLIANT

Evidence of violation: - Only ~5 explicit `_Mutation` test cases exist out of ~4,907 test functions - `test/mutation/run_mutations.sh` exists but is limited - Most packages have zero mutation coverage - Anti-bluff audit found 20/30 packages fail due to missing mutation tests

Remediation plan: 1. Add `_Mutation` tests for all 20 failing packages 2. Establish mutation test template 3. Enforce mutation test presence in gate-check 4. Target: 100% of unit tests have paired mutation tests

2.9 §7.1: Quality Guarantee

Status: ❌ NON-COMPLIANT

Evidence of violation: - PRR at 82.5% (target: 95%) - 178 orphaned packages can't deliver quality to users - PASS-bluffs exist in 8 packages - Features pass tests but don't work for end users

Remediation plan: 1. Resolve orphaned packages (wire-in/prune/document) 2. Fix all PASS-bluffs 3. Achieve PRR ≥ 95% 4. Ensure every feature works for end users

2.10 §11.4: Anti-Bluff Covenant

Status: ❌ NON-COMPLIANT

Evidence of violation: - 8 stub bluff packages identified - 20/30 packages fail anti-bluff audit - PASS-bluffs where tests pass on non-functional code - No mutation testing enforcement

Remediation plan: 1. Fix all 8 stub bluff packages 2. Add mutation tests for all failing packages 3. Enforce anti-bluff in gate-check 4. Add bluff scanner to Challenges framework

---

Chapter 3: Anti-Bluff Covenant Compliance

3.1 §11.4 Anti-Bluff Audit Results Per Package

Package	§11.4.1 FAIL-bluff	§11.4.2 Evidence	§11.4.3 Per-device	§11.4.4 Sink-side	§11.4.5 No-hardcode	Overall
`pkg/config`	❌ PASS-bluff	❌ No evidence	⚠️ N/A	❌ Returns hardcoded	❌ Hardcoded	FAIL
`pkg/grpcutil`	❌ PASS-bluff	❌ No evidence	⚠️ N/A	❌ No-op	❌ N/A	FAIL
`pkg/infra`	❌ PASS-bluff	❌ No evidence	⚠️ N/A	❌ Simulation only	❌ Fake data	FAIL
`pkg/jwt`	❌ PASS-bluff	❌ No evidence	⚠️ N/A	❌ No verification	❌ N/A	FAIL
`pkg/leader`			⚠️ N/A		❌ N/A	FAIL

Package	§11.4.1 FAIL-bluff	§11.4.2 Evidence	§11.4.3 Per-device	§11.4.4 Sink-side	§11.4.5 No-hardcode	Overall
	❌ PASS-bluff	❌ No evidence		❌ Not distributed		
`pkg/middleware`	❌ PASS-bluff	❌ No evidence	⚠️ N/A	❌ No-op	❌ N/A	FAIL
`pkg/tracing`	❌ PASS-bluff	❌ No evidence	⚠️ N/A	❌ Hardcoded IDs	❌ “trace-1”	FAIL
`pkg/websocket`	❌ PASS-bluff	❌ No evidence	⚠️ N/A	❌ Returns nil	❌ N/A	FAIL
`pkg/discovery`	✅ Real	✅ Evidence	✅ Per-platform	✅ Sink-side	✅	PASS
`pkg/errors`	✅ Real	✅ Evidence	✅ Per-platform	✅ Sink-side	✅	PASS
`pkg/log`	✅ Real	✅ Evidence	✅ Per-platform	✅ Sink-side	✅	PASS
`pkg/session`	✅ Real	✅ Evidence	✅ Per-platform	✅ Sink-side	✅	PASS
`pkg/swim`	✅ Real	✅ Evidence	✅ Per-platform	✅ Sink-side	✅	PASS

3.2 PASS-Bluff Detection Findings

Finding 1: pkg/config PASS-bluff

Description: `TestDefault` passes because it checks default values match hardcoded struct. If the struct changes, the test still passes with wrong values. **Detection:** Mutation test would catch: remove a default → test still passes. **Remediation:** Add `TestDefault_Mutation` verifying defaults are non-zero.

Finding 2: pkg/jwt PASS-bluff

Description: `TestParse` passes because it only verifies string splitting. Even a completely invalid JWT (with no real signature) passes. **Detection:** Mutation test would catch: remove signature verification (which doesn’t

exist) → test still passes. **Remediation:** Add real JWT verification and test with tampered tokens.

Finding 3: pkg/infra PASS-bluff

Description: All tests pass because they test the simulation, not real infrastructure. **Detection:** Mutation test would catch: make Boot always succeed → tests pass even if Docker is down. **Remediation:** Add integration tests against real Docker/Podman.

Finding 4: pkg/leader PASS-bluff

Description: `TestElection` passes because it only verifies an atomic flag toggles. **Detection:** Mutation test would catch: replace etcd election with atomic flag → test still passes. **Remediation:** Add integration test with real etcd election.

3.3 Mutation Testing Compliance

Package	Has _Mutation Tests	Constitution §1.1 Compliant
`pkg/discovery`	✅ 9 mutation tests	✅ COMPLIANT
`pkg/errors`	✅ 8 mutation tests	✅ COMPLIANT
`pkg/log`	✅ 8 mutation tests	✅ COMPLIANT
`pkg/session`	✅ Many mutation tests	✅ COMPLIANT
`pkg/swim`	✅ 22 mutation tests	✅ COMPLIANT
All other packages	❌ 0 mutation tests	❌ NON-COMPLIANT

Mutation testing compliance rate: 5/255 packages (2%) — far below 100% requirement.

3.4 Evidence Taxonomy Compliance

Evidence Type	Format Defined	Storage Defined	Retention Defined	Enforced
Test output	✅ Text	✅ qa-results/	❌ Not defined	❌
Coverage report	✅ HTML+JSON		❌ Not defined	❌

Evidence Type	Format Defined	Storage Defined	Retention Defined	Enforced
		✅ qa-results/coverage/		
Benchmark results	✅ JSON	✅ qa-results/benchmarks/	❌ Not defined	❌
Security scan	✅ JSON+SARIF	✅ qa-results/security/	❌ Not defined	⚠️ Partial
Challenge evidence	✅ Markdown+screenshots	✅ qa-results/challenges/	❌ Not defined	❌
Chaos results	✅ JSON+logs	✅ qa-results/chaos/	❌ Not defined	❌

Remediation plan: 1. Define retention policies for all evidence types 2. Enforce evidence collection in gate-check 3. Add evidence verification to CI pipeline 4. Implement automated evidence archival

---

Chapter 4: Cross-Platform Parity Compliance

4.1 PCS-1 Status Per Feature

Feature	Linux	macOS	Windows/WSL2	Status
CPU monitoring	✅ `/proc/stat`	✅ `sysctl`	❌	⚠️ PARTIAL
Memory monitoring	✅ `/proc/meminfo`	✅ `vm_stat`	❌	⚠️ PARTIAL
GPU detection	✅ NVIDIA SMI	✅ Apple Metal	❌	⚠️ PARTIAL
GPU monitoring	✅ DRM/sysfs	⚠️ system_profiler	❌	⚠️ PARTIAL
	✅ cgroup v2		❌ N/A	

Feature	Linux	macOS	Windows/WSL2	Status
cgroup resources		❌ N/A (no cgroups)		✅ Linux-only (justified)
WireGuard mesh	✅ kernel WG	✅ wireguard-go	❌	⚠️ PARTIAL
Power management	✅ /sys/class	✅ IOKit	❌	⚠️ PARTIAL
Health syscalls	✅ `syscall_unix`	✅ `syscall_unix`	✅ `syscall_windows`	✅ COMPLIANT
Boot coordination	✅ `linux_boot.go`	❌	❌	❌ NON-COMPLIANT
Node registration	✅ etcd	✅ etcd	❌	⚠️ PARTIAL
Session management	✅ PTY	✅ PTY	⚠️ ConPTY	⚠️ PARTIAL
Container builds	✅ Podman	✅ Podman	❌	⚠️ PARTIAL

4.2 Linux, macOS, Windows/WSL2 Coverage

Linux Coverage: ~85%

Most features work on Linux. Missing: multi-host networking, real GPU hardware testing.

macOS Coverage: ~45%

Key gaps: - No cgroup equivalent (justified — macOS has no cgroups) - DRM/GPU monitoring incomplete - WireGuard tests skipped on non-root - Boot coordination not implemented - No session testing on macOS

Windows/WSL2 Coverage: ~5%

Key gaps: - Almost no Windows-specific implementations - Only `syscall_windows.go` for health - `detect_other.go` stubs for GPU detection - No Windows session backend (ConPTY) - No Windows WireGuard support

4.3 CLAUDE-2 Retroactive Stub Tracking

Stub File	Feature	Tracking Status	Remediation
`pkg/resources/drm_other.go`	GPU monitoring	Tracked	Implement real macOS GPU probe
`pkg/resources/accel_other.go`	Accelerator monitoring	Tracked	Implement real macOS accel probe
`pkg/resources/proc_mock.go`	/proc reader	Tracked	Remove — `proc_darwin.go` exists
`pkg/gpu/detect_other.go`	GPU detection	Tracked	Implement Windows detection
`internal/gpu/detect_other.go`	GPU detection	Tracked	Implement Windows detection
`cmd/helix-setup/mem_other.go`	Memory detection	Tracked	Implement generic fallback

Chapter 5: End-User Usability Compliance

5.1 CLAUDE-1 / AGENT-1 / QWEN-1 Compliance

Overall Status: ❌ NON-COMPLIANT

Feature-by-Feature Assessment

Feature	Tests Pass?	Works for End User?	Sink-side Evidence?	CLAUDE-1 Compliant?
Node registration	✅	⚠️ Partial — etcd registration works, but	❌	❌

Feature	Tests Pass?	Works for End User?	Sink-side Evidence?	CLAUDE-1 Compliant?
		not accessible via gateway		
Session management	✅	⚠️ Partial — gRPC works, but WebSocket attach incomplete	⚠️ Partial	❌
Job scheduling	✅	❌ — Scheduler helpers not wired, no backfill	❌	❌
Build service	✅	❌ — Wrong imports bypass orchestration	❌	❌
Health checking	✅	⚠️ Partial — HTTP works, gRPC doesn’t	❌	❌
Authentication	✅	❌ — JWT stub tokens, no verification	❌	❌
RBAC	✅	❌ — Scopes defined but not enforced	❌	❌
mTLS	⚠️ Partial	❌ — SPIFFE CA works, but not wired to services	❌	❌
WireGuard mesh	✅	⚠️ Partial — Works on Linux+root only	⚠️ Partial	❌
LLM inference	✅	❌ — No actual LLM	❌	❌

Feature	Tests Pass?	Works for End User?	Sink-side Evidence?	CLAUDE-1 Compliant?
		backend connected		
Federation	✅	❌ — Not wired into any binary	❌	❌
Marketplace	✅	❌ — Not wired into any binary	❌	❌
GPU management	✅	⚠️ Partial — NVIDIA works, others missing	⚠️ Partial	❌
Console hardware	✅	❌ — Detection works but not integrated	❌	❌
CRDT sessions	✅	✅ — Real CRDT with LWW	✅	✅
SWIM gossip	✅	✅ — Real SWIM protocol	✅	✅
Policy engine	✅	✅ — Real OPA engine	✅	✅

Summary

- **CLAUDE-1 Compliant features:** 3/17 (CRDT sessions, SWIM gossip, Policy engine)
- **CLAUDE-1 Non-compliant features:** 14/17
- **Primary causes:** Orphaned code, stub bluffs, incomplete wiring

5.2 Sink-Side Evidence for Each Feature

Feature	Sink-Side Evidence Available?	What's Missing
Node registration	❌	

Feature	Sink-Side Evidence Available?	What's Missing
		No verification that node appears in gateway API
Session management	⚠️	CRDT state verified, but WebSocket attach not verified
Job scheduling	❌	No verification that scheduled job actually runs
Build service	❌	No verification that built artifact is valid
Health checking	❌	No verification that health updates propagate to gateway
Authentication	❌	No verification that auth tokens are actually validated
RBAC	❌	No verification that unauthorized access is denied
mTLS	❌	No verification that traffic is actually encrypted
WireGuard	⚠️	Connectivity verified on Linux, not on macOS

5.3 Mock-Only Validation Audit

Package	Uses Mocks?	Mock Justified? (§11.4.27)	Alternative
`pkg/infra`	✅ Entire package is simulation	❌ — Claims orchestration	Real Docker/ Podman backend
`pkg/leader`	✅ In-memory only	❌ — Claims distributed election	etcd election backend

Package	Uses Mocks?	Mock Justified? (§11.4.27)	Alternative
`pkg/jwt`	✅ No real JWT verification	❌ — Claims JWT parsing	golang-jwt/jwt/v5 library
`pkg/tracing`	✅ Hardcoded IDs	❌ — Claims distributed tracing	OpenTelemetry integration
`pkg/grpcutil`	✅ No-op interceptors	❌ — Claims interceptor functionality	Real logging/metrics/auth
`pkg/middleware`	✅ No-op middleware	❌ — Claims logging middleware	Real request logging
`pkg/config`	✅ Hardcoded config	❌ — Claims env/file loading	Real env/file config
`pkg/websocket`	✅ Nil upgrade	❌ — Claims WebSocket support	gorilla/websocket library

All 8 stub bluff packages use mock-only validation that violates §11.4.27 (mocks permitted ONLY in unit tests).

Chapter 6: Documentation Sync Compliance

6.1 CLAUDE-3 / AGENT-2 / QWEN-2 Compliance

Overall Status: ⚠️ PARTIAL

docs_chain Integration Status

Component	Status	Details
`.docs_chain/contexts/tracked_docs.yaml`	✅ Exists	Tracked documentation contexts
	✅ Exists	Gap audit contexts

Component	Status	Details
`.docs_chain/contexts/gap_audits.yaml`		
`scripts/docs/generate.sh`	✅ Exists	Documentation generation
`scripts/docs/verify.sh`	✅ Exists	Documentation verification
`scripts/docs/run_docs_chain.sh`	✅ Exists	docs_chain execution
`scripts/docs/update_continuation.sh`	✅ Exists	Continuation updates
`scripts/docs/db_to_md.py`	✅ Exists	DB to Markdown conversion
`scripts/docs/update_continuation.py`	✅ Exists	Continuation Python script
docs_chain sync enforced per wave	⚠️ Partial	Not always run with every change
All materials registered	❌	Not all docs registered
Export formats generated	❌	md → html/pdf/docx not verified
Pre-build gate	❌	docs_chain verify not enforced

Documentation Coverage Assessment

Material	In Sync?	Registered in docs_chain?	Export Formats?
Main README	⚠️ May be stale	✅	❌ Not verified
CLAUDE.md	✅ Current	✅	❌ Not verified
AGENTS.md	✅ Current	✅	❌ Not verified
Architecture docs	⚠️ May be stale	⚠️ Partial	❌ Not verified
User guide	⚠️ May be stale	❌ Not registered	❌ Not verified

Material	In Sync?	Registered in docs_chain?	Export Formats?
SQL schema docs	⚠️ Drift issue (HXC-1639)	❌ Not registered	❌ Not verified
API documentation	⚠️ OpenAPI incomplete	❌ Not registered	❌ Not verified
Changelog	⚠️ May be stale	✅	❌ Not verified
ADRs	✅ Current	❌ Not registered	❌ Not verified

Remediation Plan

1. Register all materials in `.docs_chain/contexts/`
2. Add `docs_chain verify` to pre-build gate
3. Ensure all export formats are generated
4. Run docs_chain sync with every change
5. Fix SQL schema documentation drift (HXC-1639)
6. Complete OpenAPI specification
7. Generate and verify all exports (md → html/pdf/docx)

Compliance Summary

Overall Compliance Score

Rule Category	Total Rules	Compliant	Partial	Non-Compliant	Score
PCS Rules	12	2	5	5	37.5%
CLAUDE Rules	3	0	2	1	33.3%
AGENT Rules	2	0	1	1	25.0%
QWEN Rules	2	0	1	1	25.0%
§1.1 (Mutation)	1	0	0	1	0.0%
§7.1 (Quality)	1	0	0	1	0.0%

Rule Category	Total Rules	Compliant	Partial	Non-Compliant	Score
§11.4 (Anti-bluff)	5	0	1	4	10.0%
Overall	**26**	**2**	**10**	**14**	**19.2%**

Critical Non-Compliance Items

#	Rule	Violation	Impact	Priority
1	CLAUDE-1	178 orphaned packages	Features can’t be used	P0
2	§11.4	8 stub bluff packages	Non-functional features	P0
3	CLAUDE-1	CRIT-1: JWT stubs	Security bypass	P0
4	§1.1	No mutation tests	PASS-bluffs undetected	P0
5	PCS-1	No Windows support	Cross-platform gap	P1
6	PCS-2	Missing AMD/Intel GPU	GPU backend gap	P1
7	CLAUDE-3	Docs not fully synced	Documentation drift	P1
8	§7.1	PRR at 82.5%	Below 95% bar	P1

End of Constitutional Compliance Report

Document Statistics: - Total rules extracted: 26 - Total compliance assessments: 26 - Total anti-bluff audits per package: 13 - Total cross-platform assessments: 12 features - Total end-user usability assessments: 17 features - Total documentation coverage assessments: 9 materials